



# Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

## EDS Assignment no.2

Done by,  
Name: Gayatri Ravindra Devalekar  
Division: CS3  
Batch: C34

### Project Report: Paper Review Data Analysis using NumPy & Pandas:

#### 1. Introduction

This project focuses on the computational analysis of a real-world dataset containing peer reviews for papers submitted to international conferences. Peer review is a critical process in academia that ensures the quality and validity of scientific research. By applying data science techniques, we aim to uncover patterns in reviewer behavior, scoring consistency, and the relationship between reviewer confidence and final decisions.

#### 2. Dataset Description

The dataset used in this study is the **Paper Reviews Data Set**, originally curated by Keith (2017) and hosted by the UCI Machine Learning Repository.

- **Source:** UCI Machine Learning Repository / Kaggle
- **Format:** The raw data is structured in **JSON** format, representing a hierarchical relationship between papers and their multiple associated reviews.
- **Key Features:**
  - **Paper ID:** Unique identifier for each submission.
  - **Evaluation Score:** A numeric value representing the reviewer's judgment.
  - **Confidence:** A self-reported metric of the reviewer's expertise in the specific topic.
  - **Review Text:** Qualitative feedback provided by the reviewer.
  - **Preliminary Decision:** The initial acceptance or rejection status of the paper.

```

import pandas as pd
import numpy as np
import kagglehub
import os
import json

# --- SETUP: DATA LOADING & CLEANING ---
path = kagglehub.dataset_download("fnbalves/paper-reviews-data-set")
files = [f for f in os.listdir(path) if f.endswith('.json')]
with open(os.path.join(path, files[0]), 'r') as f:
    data = json.load(f)

papers_list = data['paper'] if isinstance(data, dict) and 'paper' in data else data
df = pd.json_normalize(
    papers_list,
    record_path='review',
    meta=['id', 'preliminary_decision'],
    record_prefix='rev_',
    meta_prefix='paper_',
    errors='ignore'
)

# Standardizing column names
df.rename(columns={
    'rev_evaluation': 'score',
    'rev_confidence': 'confidence',
    'paper_id': 'paper_id',
    'rev_text': 'review_text',
    'paper_preliminary_decision': 'decision'

```

```

}, inplace=True)

# Ensure numeric types
df['score'] = pd.to_numeric(df['score'], errors='coerce')
df['confidence'] = pd.to_numeric(df['confidence'], errors='coerce')
df = df.dropna(subset=['score', 'confidence'])

print("--- DATASET READY. EXECUTING PROBLEM STATEMENTS ---\n")

```

... Using Colab cache for faster access to the 'paper-reviews-data-set' dataset.  
 --- DATASET READY. EXECUTING PROBLEM STATEMENTS ---

## □ Problem Statements:

### 1. Impute missing review text with 'No review provided':

```

print("1. Impute missing review text with 'No review provided':")
df['review_text'] = df['review_text'].fillna("No review provided")
print(df['review_text'].head(3), "\n")

```

```

1. Impute missing review text with 'No review provided':
0    - El artículo aborda un problema contingente y...
1    El artículo presenta recomendaciones prácticas...
2    - El tema es muy interesante y puede ser de mu...
Name: review_text, dtype: object

```

## 2. Descriptive Stats using NumPy (Mean and Std Dev of Scores):

```
print("2. Descriptive Stats using NumPy (Mean and Std Dev of Scores):")
print(f"Mean: {np.mean(df['score']):.2f}, Std Dev: {np.std(df['score']):.2f}")
```

```
2. Descriptive Stats using NumPy (Mean and Std Dev of Scores):
Mean: 0.19, Std Dev: 1.50
```

## 3. Filtering: Show first 3 reviews with Confidence >= 4:

```
print("3. Filtering: Show first 3 reviews with Confidence >= 4:")
print(df[df['confidence'] >= 4][['paper_id', 'confidence']].head(3), "\n")
```

```
... 3. Filtering: Show first 3 reviews with Confidence >= 4:
   paper_id  confidence
0         1         4.0
1         1         4.0
2         1         5.0
```

## 4. Feature Engineering: Word Count of reviews:

```
print("4. Feature Engineering: Word Count of reviews:")
df['word_count'] = df['review_text'].str.split().str.len()
print(df[['review_text', 'word_count']].head(3), "\n")
```

```
4. Feature Engineering: Word Count of reviews:
   review_text  word_count
0 - El artículo aborda un problema contingente y...      93
1 El artículo presenta recomendaciones prácticas...      94
2 - El tema es muy interesante y puede ser de mu...     211
```

## 5. Logic Mapping: Acceptance Probability based on score:

```
print("5. Logic Mapping: Acceptance Probability based on score:")
df['accept_prob'] = np.where(df['score'] > 0, 'High', 'Low')
print(df[['score', 'accept_prob']].head(3), "\n")
```

```
5. Logic Mapping: Acceptance Probability based on score:
   score  accept_prob
0      1         High
1      1         High
2      1         High
```

## 6. Grouping: Average Score for the first 5 Papers:

```
print("6. Grouping: Average Score for the first 5 Papers:")
print(df.groupby('paper_id')['score'].mean().head(5), "\n")
```

```
6. Grouping: Average Score for the first 5 Papers:
paper_id
1      1.000000
2      2.000000
3      1.333333
4      0.000000
5      2.000000
Name: score, dtype: float64
```

## 7. Confidence Binning: Categorize reviewers:

```
print("7. Confidence Binning: Categorize reviewers:")
conds = [df['confidence'] <= 2, df['confidence'] == 3, df['confidence'] > 3]
labels = ['Low', 'Medium', 'High']
df['conf_label'] = np.select(conds, labels, default='Unknown')
print(df[['confidence', 'conf_label']].head(5), "\n")
```

```
7. Confidence Binning: Categorize reviewers:
confidence conf_label
0          4.0      High
1          4.0      High
2          5.0      High
3          4.0      High
4          4.0      High
```

## 8. Statistical Outliers: Reviews more than 2 Std Dev from mean:

```
print("8. Statistical Outliers: Reviews more than 2 Std Dev from mean:")
mean_s, std_s = df['score'].mean(), df['score'].std()
outliers = df[np.abs(df['score'] - mean_s) > (2 * std_s)]
print(f"Found {len(outliers)} outliers.\n")
```

```
8. Statistical Outliers: Reviews more than 2 Std Dev from mean:
Found 0 outliers.
```

## 9. Sorting: Top 3 most controversial papers (highest score variance):

```
print("9. Sorting: Top 3 most controversial papers (highest score variance)")
print(df.groupby('paper_id')['score'].std().sort_values(ascending=False).
```

```
9. Sorting: Top 3 most controversial papers (highest score variance):
paper_id
7      2.828427
4      2.828427
135    2.828427
Name: score, dtype: float64
```

## 10. Pattern Matching: Reviews containing 'methodology':

```
print("10. Pattern Matching: Reviews containing 'methodology':")
print(f"Count: {len(df[df['review_text'].str.contains('methodology'), case
```

```
10. Pattern Matching: Reviews containing 'methodology':
Count: 5
```

## 11. Min-Max Normalization of Scores (0 to 1):

```
print("11. Min-Max Normalization of Scores (0 to 1):")
s_min, s_max = df['score'].min(), df['score'].max()
df['norm_score'] = (df['score'] - s_min) / (s_max - s_min)
print(df[['score', 'norm_score']].head(3), "\n")
```

```
11. Min-Max Normalization of Scores (0 to 1):
   score  norm_score
0      1         0.75
1      1         0.75
2      1         0.75
```

## 12. Correlation: Relationship between Word Count and Score:

```
print("12. Correlation: Relationship between Word Count and Score:")
print(f"Correlation Coefficient: {df['word_count'].corr(df['score']):.4f}")
```

```
12. Correlation: Relationship between Word Count and Score:
Correlation Coefficient: -0.2604
```

### 13. Data Integrity: Total number of unique papers:

```
print("13. Data Integrity: Total number of unique papers:")
print(f"Unique Papers: {df['paper_id'].nunique()}\n")
```

```
13. Data Integrity: Total number of unique papers:
Unique Papers: 169
```

### 14. Random Sampling: Display 3 random reviews:

```
print("14. Random Sampling: Display 3 random reviews:")
print(df[['paper_id', 'review_text']].sample(3), "\n")
```

```
14. Random Sampling: Display 3 random reviews:
   paper_id  review_text
243      106  En su forma, tiene algunos problemas, por ejem...
357      149  El artículo presenta una aproximación por Algo...
174       78  -\tEl tema es de relevancia científica. -\tEn ...
```

### 15. Cross-tabulation: Decision vs Confidence Label:

```
print("15. Cross-tabulation: Decision vs Confidence Label:")
print(pd.crosstab(df['decision'], df['conf_label']), "\n")
```

```
15. Cross-tabulation: Decision vs Confidence Label:
conf_label      High  Low  Medium
decision
accept           138   24    100
no decision         1    0     0
probably reject     10    2     6
reject             75    7    40
```

### 16. Extreme Values: Details of the review with the highest score:

```
print("16. Extreme Values: Details of the review with the highest score:")
print(df.loc[df['score'].idxmax()][['paper_id', 'score']], "\n")
```

```
16. Extreme Values: Details of the review with the highest score:
paper_id    2
score       2
Name: 3, dtype: object
```

## 17. Column Removal: Dropping 'rev\_lan' if present:

```
print("17. Column Removal: Dropping 'rev_lan' if present:")
df_dropped = df.drop(columns=[col for col in ['rev_lan'] if col in df.col
print(f"Columns remaining: {list(df_dropped.columns[:5])}...\n")
```

```
17. Column Removal: Dropping 'rev_lan' if present:
Columns remaining: ['confidence', 'score', 'rev_id', 'rev_orientation', 'r
```

## 18. Decision Encoding: Mapping 'accept' to 1 and 'reject' to 0:

```
print("18. Decision Encoding: Mapping 'accept' to 1 and 'reject' to 0:")
df['decision_int'] = df['decision'].map({'accept': 1, 'reject': 0}).fillna
print(df[['decision', 'decision_int']].head(3), "\n")
```

```
18. Decision Encoding: Mapping 'accept' to 1 and 'reject' to 0:
  decision  decision_int
0  accept             1.0
1  accept             1.0
2  accept             1.0
```

## 19. Aggregation: Median confidence for each unique score:

```
▶ print("19. Aggregation: Median confidence for each unique score:")
print(df.groupby('score')['confidence'].median(), "\n")
```

```
... 19. Aggregation: Median confidence for each unique score:
score
-2    4.0
-1    4.0
0     4.0
1     3.0
2     4.0
Name: confidence, dtype: float64
```

## 20. Percentile Slicing: Word count threshold for the longest 10% of reviews:

```
print("20. Percentile Slicing: Word count threshold for the longest 10% o  
threshold = np.percentile(df['word_count'], 90)  
print(f"90th Percentile Word Count: {threshold} words\n")
```

```
20. Percentile Slicing: Word count threshold for the longest 10% of review  
90th Percentile Word Count: 312.6 words
```

## Conclusion:

The analytical processing of the Paper Reviews dataset demonstrates the power of Python's data science ecosystem in transforming semi-structured JSON data into structured insights. By leveraging Pandas for complex data manipulation and NumPy for high-performance vectorized operations, we successfully automated the extraction of reviewer patterns and scoring trends.

The project highlights that while raw data is often messy and nested, systematic cleaning—such as normalization, type casting, and outlier detection—is the essential bridge between raw information and meaningful academic analysis. This workflow serves as a scalable foundation for more advanced research into peer-review objectivity and automated decision-making.